

CRM Refactor Deployment Brief

Everything changed in this session, what it means for the production database, and the order it must be applied in.

DATE	BASE	FILES CHANGED	MIGRATIONS
24 August 2026	master @ ffa069af	39	2

Contents

1	Read this first	6	Database changes
2	What shipped — three workstreams	7	Production runbook
3	Workstream A — CRM bug fixes	8	Rollback
4	Workstream B — Lead assignment	9	Deferred and open items
5	Workstream C — Identity reprint	10	File inventory

1 Read this first

This application applies migrations automatically on startup. `API/Program.cs:194` calls `context.Database.MigrateAsync()`, followed by `SeedContracts.SeedData(...)` at line 198. **Deploying the code IS applying the database changes.** There is no separate "apply migrations" decision — the moment the production service restarts on this build, both migrations in section 6 run.

This is how the two migrations reached the development database. Plan the deployment around it.

One SQL script must run in the same maintenance window as the deploy. `grant_crm_lead_assignment_roles_2026_08_24.sql`. Without it, **every CRM user in production opens the Leads screen and sees nothing at all.** Not an error — an empty grid. This is currently the live state of the development environment, because the script has not been run there either.

Cause: lead visibility is now scoped by the `CRM_Leads_ViewAll` role. The role is created automatically by the seeder but **granted to nobody**. No lead has an owner yet, so a user without that role matches zero rows.

Everyone must sign out and back in. Security roles are baked into the JWT at login and the token lives for 7 days. Until a user re-authenticates, the new roles do not apply to them and they will still see an empty leads list.

2 What shipped — three workstreams

	Workstream	Summary	DB impact
A	CRM bug fixes	Three defects found in the module review: a dead route, opportunity fields silently discarded on save, and cancelled deals never closing	None

	Workstream	Summary	DB impact
B	Lead assignment	New capability. Assign, reassign, unassign and bulk-assign leads to sales reps; ownership history; role-gated so only CRM Admins can distribute work	1 migration
C	Identity repoint	Four audit columns moved from the OFBiz <code>USER_LOGIN</code> table to <code>AspNetUsers</code> , because <code>USER_LOGIN</code> covers only 10 of 24 users	1 migration

A and B are additive. C is a correctness fix that also unblocks B — without it, assignments made by most CRM staff would have recorded no actor, and one of the two designated CRM Admins could not have created an opportunity at all.

3 Workstream A — CRM bug fixes

3.1 Dead `/crm` route removed

`CRMDashboard.tsx` imported a component that does not exist (`../form/CreateContractorForm` — the real file lives under `features/parties/`), rendered a hard-coded empty grid, and had a "Create New Lead" button that only set an unused state variable. Nothing linked to it: the header points at `/sales-opportunities` and the CRM sub-menu never referenced it.

Removed the component and its route. `/leads` and `/sales-opportunities` are unaffected and still gated behind `CRM_View`.

3.2 Opportunity fields were being discarded

`CreateSalesOpportunity` persisted only stage, project and unit onto the opportunity row. Name, description, estimated amount, currency, probability, close date, next step, data source, campaign and type were written to the *history* row only and lost from the record itself — which is why the board's value and close-date fields never displayed anything.

Both create and update now persist the full payload. Every assignment on the update path is null-guarded, because the board's drag-and-drop stage change reuses the same command with a sparse DTO — an unguarded write there would have blanked the record.

Not fixed: the opportunity form still does not collect amount, close date or name. Those fields now persist correctly when supplied by the API or an import, but no UI captures them, and the board's currency chip remains commented out. The pipeline still carries no monetary value.

3.3 Cancelled deals never closed

Only `DONE_DEAL` had any effect, and even that set `IS_WON` without `IS_CLOSED`. A cancelled opportunity sat in its original pipeline column indefinitely.

Cancellation actions now close the opportunity, move it to `SOSTG_CLOSED_LOST`, zero the probability and write a history entry naming the cancellation reason. Done Deal now sets both flags. The same flags are applied when a card is dragged to a closed column, and **cleared** when dragged back to an open stage — without that, a reopened deal would keep its action form locked forever.

Known residual: winning a deal sets the apartment to `APARTMENT_RESERVED`, but nothing ever releases it. Reopening or cancelling leaves the unit reserved. Pre-existing behaviour, not introduced here.

4 Workstream B — Lead assignment

Previously there was no way to assign a lead to anyone. A 500-row import landed with every lead belonging to nobody, and no screen or field could change that. The only ownership in the module was on the *opportunity*, set once a deal already existed — far downstream of where work is actually distributed.

4.1 Data model — no new table

`PartyRelationshipType` already contained a seeded `LEAD_OWNER` row, unused, and `PARTY_RELATIONSHIP` was empty. One active assignment is one row:

Column	Value
<code>PARTY_ID_FROM</code> / <code>ROLE_TYPE_ID_FROM</code>	the sales rep / <code>SALES_REP</code>

Column	Value
PARTY_ID_TO / ROLE_TYPE_ID_TO	the lead / LEAD
PARTY_RELATIONSHIP_TYPE_ID	LEAD_OWNER
FROM_DATE / THRU_DATE	assignment window; THRU_DATE null while active

Reassignment closes the open row and opens a new one, so ownership history comes for free. Unassigning closes the row rather than deleting it.

4.2 What users get

- **Assign / reassign / unassign** from a row action on the leads grid, with the current owner shown
- **Bulk assign** — checkbox selection with select-all-on-page, and a toolbar action. Selection survives paging, because it is held as a set of ids rather than a flag on the row
- **Assign on import** — after a batch import, a prompt offers to assign the newly created leads to a rep.
`CreateLeadsBatch` now returns the ids it created so the UI can do this; creation itself still never assigns
- **Owner column** on the grid, sortable and filterable server-side
- **Ownership history** in the assign dialog — who held the lead, over what window, with the assignment note

Partial-success semantics on bulk assign mirror the existing batch import: per-lead failure reasons, and leads already owned by the target rep are counted separately rather than written twice.

4.3 Two new roles

Role	Gates
CRM_Leads_Assign	Assign, reassign, unassign, bulk assign, and read assignment history. The CRM Admin's defining permission. Enforced with <code>[Authorize(Roles=...)]</code> on all four endpoints
CRM_Leads_ViewAll	See every lead. Without it a user sees only leads assigned to them. Enforced inside <code>ListLeads</code> as a filter on the <code>IQueryable</code> , so it applies before OData paging — a user cannot page or filter around it

Why this matters beyond these two roles. Before this change, no granular CRM role was read anywhere in the code. `CRM_View` gated the whole module at the route level and nothing else was checked, so all 13 CRM users were functionally identical despite holding nine roles each. These two are the first CRM permissions that actually do something.

Scoping fails closed: a user who cannot be resolved to a party sees nothing rather than everything. Unassigned leads are visible only to `ViewAll` holders, which is what makes distribution an admin function.

5 Workstream C — Identity repoint

5.1 The problem

Four audit columns recorded "who did this" as a foreign key into the OFBiz `USER_LOGIN` table. That table is a hard-coded seed list in `SeedContracts.cs`, is not used for authentication, and is only partially populated.

Measure	Development
Rows in <code>USER_LOGIN</code>	10
Rows in <code>AspNetUsers</code>	24
CRM users with a <code>USER_LOGIN</code> row	2 of 13 (Emad, Ahmad)

The consequence was silent data loss: an action or history entry written by any of the eleven CRM sales staff recorded a **null actor**, and the history screen displayed a raw GUID instead of a name. For the new assignment feature it was worse — one of the two designated CRM Admins (`rmostafa@gmail.com`) has no `USER_LOGIN` row, so every assignment they made would have recorded nothing, and the foreign key would have rejected any attempt to store their real identity.

5.2 The change

All four columns now reference `AspNetUsers(Id)` — the table the application actually authenticates against.

Table	Column	New constraint
<code>PARTY_RELATIONSHIP</code>	<code>CREATED_BY_USER_LOGIN</code>	<code>PARTY_REL_CREATED_BY_USER</code>
<code>SALES_OPPORTUNITY</code>	<code>CREATED_BY_USER_LOGIN</code>	<code>SLSOPP_CREATED_BY_USER</code>
<code>SALES_OPPORTUNITY_ACTION</code>	<code>CREATED_BY_USER_LOGIN</code>	<code>SLSOPPACT_CREATED_BY_USER</code>
<code>SALES_OPPORTUNITY_HISTORY</code>	<code>MODIFIED_BY_USER_LOGIN</code>	<code>SLOPHI_MODIFIED_BY_USER</code>

Column names are unchanged, so no application code outside these handlers is affected. `IUserAccessor` gained `GetUserId()`, which reads the id straight off the JWT's `NameIdentifier` claim. The old three-hop lookup (username → `PartyId` → `UserLoginId`) is gone: **two database round-trips per write become zero**, and the history screen's name lookup collapsed from three queries to one that works for all 24 users.

`SALES_OPPORTUNITY_ACTION.CREATED_BY_USER_LOGIN` also became **nullable**. It was previously `NOT NULL` with the handler falling back to the literal string `"SYSTEM"`, which is not a valid `AspNetUsers.Id` and would have failed the new foreign key outright. An unresolvable actor now records nothing rather than a fake user.

6 Database changes

6.1 Migrations (applied automatically on startup)

Migration	Effect
<code>20260824080824_AddCreatedByUserLoginToPartyRelationship</code>	Adds nullable <code>CREATED_BY_USER_LOGIN</code> to <code>PARTY_RELATIONSHIP</code> with an index and FK
<code>20260824094531_RepointActorColumnsToAspNetUsers</code>	Drops four <code>USER_LOGIN</code> FKs, widens the four columns to <code>varchar(255)</code> , adds four FKs to <code>AspNetUsers(Id)</code>

Both are already applied in development. Equivalent raw SQL is checked in under `API/Sql/Production/` for reference, but running it is unnecessary if the service starts normally — and would fail if run afterwards, since the constraints would

already exist.

6.2 No data migration required

Every existing actor value in development is already a valid `AspNetUsers.Id`, because the only two people who have used the CRM are among the ten seeded `USER_LOGIN` rows — whose ids were set to match their ASP.NET identity. **This must be re-verified on production, where other users may have written rows.** See runbook step 2.

6.3 Reference data prerequisite

`LEAD_OWNER` must exist in `PARTY_RELATIONSHIP_TYPE`, or every assignment fails on the foreign key. Confirmed present in production.

7 Production runbook

Steps are ordered. Do not start the service before step 3.

Action	Expected / notes	✓
1 Back up the production database, or at minimum <code>AspNetUserRoles</code> , <code>PARTY_RELATIONSHIP</code> , <code>SALES_OPPORTUNITY</code> , <code>SALES_OPPORTUNITY_ACTION</code> , <code>SALES_OPPORTUNITY_HISTORY</code>	Migrations are applied by the app and are not trivially reversible in place	☐
2 Pre-flight data check. Run the three queries below	All three must return 0 . A non-zero result is a stale <code>USER_LOGIN_ID</code> that will fail the new foreign key — null it out before continuing	☐
3 Confirm <code>LEAD_OWNER</code> exists	<code>SELECT * FROM PARTY_RELATIONSHIP_TYPE WHERE PARTY_RELATIONSHIP_TYPE_ID='LEAD_OWNER';</code> — 1 row	☐
4 Deploy the build and start the service	Startup applies both migrations and creates the two new roles. Watch the log for "Migration done" and "Seeding done"	☐
5 Verify the migrations landed	<code>SELECT MigrationId FROM __EFMigrationsHistory ORDER BY MigrationId DESC LIMIT 2;</code> — both from section 6.1	☐
6 Run <code>grant_crm_lead_assignment_roles_2026_08_24.sql</code>	CRITICAL Grants <code>CRM_Leads_Assign</code> to the two CRM Admins and <code>CRM_Leads_ViewAll</code> to every <code>CRM_View</code> holder. Its PART 1 is read-only — inspect before applying PART 2	☐
7 Run the script's PART 3 verification	Query 3c must return 0 rows . It lists anyone who can reach CRM but would see an empty leads list	☐
8 Tell all CRM users to sign out and back in	Roles live in the JWT; the token lasts 7 days	☐
9 Smoke test — see 7.2		☐

Step 2 — pre-flight data check

```
SELECT 'ACTION' AS t, COUNT(*) AS bad FROM SALES_OPPORTUNITY_ACTION a
  WHERE a.CREATED_BY_USER_LOGIN IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM AspNetUsers u WHERE u.Id = a.CREATED_BY_USER_LOGIN)
UNION ALL SELECT 'HISTORY', COUNT(*) FROM SALES_OPPORTUNITY_HISTORY h
  WHERE h.MODIFIED_BY_USER_LOGIN IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM AspNetUsers u WHERE u.Id = h.MODIFIED_BY_USER_LOGIN)
UNION ALL SELECT 'OPPORTUNITY', COUNT(*) FROM SALES_OPPORTUNITY s
  WHERE s.CREATED_BY_USER_LOGIN IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM AspNetUsers u WHERE u.Id = s.CREATED_BY_USER_LOGIN);
```

Production is likelier than development to contain rows written by the eleven CRM staff who have no `USER_LOGIN` entry — but those recorded *null*, not a bad id, so they pass. A non-zero count means a real `USER_LOGIN_ID` that does not match any ASP.NET user.

7.2 Smoke test after deployment

Check		Expected	✓
S1	Sign in as a CRM Admin, open Leads	Leads are visible; an "Assigned To" column shows "Unassigned"	☐
S2	Sign in as a regular CRM user, open Leads	Leads visible (they hold <code>ViewAll</code> after step 6); no Assign button or checkboxes	☐
S3	As Admin, assign one lead to a rep	Owner appears in the grid; ownership history shows one entry marked Current	☐
S4	Reassign that lead to a different rep	History shows two entries; the first now has an end date	☐
S5	Confirm the actor was recorded	<code>SELECT CREATED_BY_USER_LOGIN FROM PARTY_RELATIONSHIP WHERE THRU_DATE IS NULL;</code> — a real <code>AspNetUsers.Id</code> , not null	☐
S6	Log an action on an opportunity, then open its History tab	Entries show a person's name , not a GUID — this is the visible proof of workstream C	☐
S7	Log a Cancellation on an open opportunity	Moves to Closed Lost, locks, history note names the reason	☐
S8	Drag a card between stages on the board	Stage changes; the opportunity's other fields are not blanked	☐
S9	Browse to <code>/crm</code>	Route no longer exists; no blank page or console error	☐

8 Rollback

The database changes are not cleanly reversible by restarting an older build. EF does not un-apply migrations on startup — rolling the code back leaves the schema forward. Plan to roll forward, or restore from the step 1 backup.

8.1 Reversing the roles only

The least destructive intervention. If the leads screen misbehaves, granting `CRM_Leads_ViewAll` to everyone restores pre-deployment visibility without touching schema. The grant script takes its own backup table (`AspNetUserRoles_bak_20260824`) and documents the restore statement.

8.2 Reversing the schema

Each migration has a generated `Down`. One caveat on `RepointActorColumnsToAspNetUsers`: reverting restores `SALES_OPPORTUNITY_ACTION.CREATED_BY_USER_LOGIN` to `NOT NULL`, so EF first rewrites any null actor to an **empty string** — and `''` is not a valid `USER_LOGIN_ID`, so the restored foreign key would then reject those rows. EF flagged this at scaffold time as a possible data-loss operation. Treat the `Down` as a last resort.

8.3 MySQL does not do transactional DDL

The generated scripts wrap statements in `START TRANSACTION ... COMMIT`, but each `ALTER` commits implicitly. If a later statement fails, earlier ones are already applied. This is precisely why the step 2 pre-flight check matters.

9 Deferred and open items

Status	Item	Detail
DEFERRED	Revoke <code>CRM_Leads_ViewAll</code> from the ten regular CRM users	The final step that makes the admin/regular split real. Must wait until leads have actually been assigned to those reps — running it sooner hands them an empty screen. No script written yet
DEFERRED	Drop Ahmad's CRM roles	Script exists and is unused: <code>drop_crm_roles_from_ahmad_2026_08_23.sql</code> . Decision was to leave his access in place for now
OPEN	Seeder did not grant the new roles	The two roles were added to <code>eradwan1967@gmail.com</code> 's block in <code>SeedContracts.cs</code> , but after startup zero users hold them . Not understood yet. Until it is, do not rely on seeding to grant roles — use the SQL script
OPEN	Unassign records no actor	Unassigning closes a row rather than creating one, so there is nowhere to store the acting user. You can see who assigned a lead, never who removed it. Needs a second column or a log table
OPEN	Units are never released	Winning a deal reserves the apartment; cancelling or reopening never returns it to available
MINOR	Duplicate FK registration	<code>SalesOpportunityAction</code> configures its actor foreign key twice in <code>DataContext.cs</code> , with different indentation. Pre-existing and harmless to EF, but it caused a build failure during this work and is worth tidying
MINOR	Opportunity form captures no money	See 3.2 — persistence is fixed, the UI is not

10 File inventory

39 files changed. The large line count is dominated by the EF `.Designer.cs` snapshot, which is generated and not hand-written.

New — backend

```
Application/CRM/Leads/Assignment/AssignLead.cs
Application/CRM/Leads/Assignment/BulkAssignLeads.cs
Application/CRM/Leads/Assignment/UnassignLead.cs
Application/CRM/Leads/Assignment/ListLeadAssignmentHistory.cs
Application/CRM/Leads/Assignment/LeadAssignmentDto.cs
Application/CRM/Leads/Assignment/LeadAssignmentConstants.cs
Persistence/Migrations/20260824080824_AddCreatedByUserLoginToPartyRelationship.cs
Persistence/Migrations/20260824094531_RepointActorColumnsToAspNetUsers.cs
```

New — frontend

```
client-app/src/features/CRM/leads/AssignLeadModal.tsx
client-app/src/features/CRM/leads/BulkAssignLeadsModal.tsx
```

New — SQL (in `API/Sql/Production/`)

Script	Status
grant_crm_lead_assignment_roles_2026_08_24.sql	Must run — runbook step 6
crm_admin_roles_and_copy_nevine_to_rmostafa_2026_08_23.sql	Already applied, dev and prod (step 2b deliberately skipped)
drop_crm_roles_from_ahmad_2026_08_23.sql	Deferred, not run anywhere
repoint_actor_columns_to_aspnetusers_2026_08_24.sql	Reference only — the app applies this migration itself
add_created_by_user_login_to_party_relationship_2026_08_24.sql	Reference only — already applied

Modified — notable

File	Change
Application/CRM/Leads/ListLeads.cs	Owner projection + <code>ViewAll</code> scoping
API/Controllers/CRM/LeadsController.cs	4 endpoints, all role-gated
Application/Interfaces/IUserAccessor.cs	<code>GetUserId()</code> , <code>IsInRole()</code>
Persistence/DataContext.cs	4 FKs repointed; orphaned inverse collections removed
Domain/UserLogin.cs	Two now-orphaned collections removed — left in place, EF would have invented shadow FK columns
Persistence/SeedContracts.cs	Two new roles registered
client-app/src/features/CRM/leads/LeadsList.tsx	Owner column, selection, assign actions
client-app/src/features/CRM/leads/ImportedDataGrid.tsx	Assign-on-import step

Development environment at time of writing: 1 lead, 2 opportunities, 2 actions, 16 history rows, 0 lead assignments, 10 `USER_LOGIN` rows, 24 `AspNetUsers`. Both migrations applied; the grant script has **not** been run, so all 13 CRM users currently see an empty leads list there. Correcting that in development is the same step 6 as in production.